

Visão geral do fluxo

Carregamento de dados:

df = arquivo cru da base de dados feita para regressão

```
def agregar(df, servico):
```

Função de agregação das variáveis utilizadas para cada tipo de serviço

Parâmetros:

df = fonte de dados que estará sendo consultada

servico = qual tipo de serviço (validação, formatação ou expansão)

retorna a variável g = tabela de agregação com duas novas colunas de margem em pct e o número do mês isolado

```
def rodar_ols(agg, variaveis):
```

Função que roda o modelo em cima das variáveis respectivas de cada tipo de serviço,

Parâmetros:

agg = tabela agregada para rodar no modelo

variáveis = variáveis que estarão presentes no modelo

retorna = m1 (modelo de custos [.fit]), m2 (modelo de margem de lucro % [.fit]), y1 (estimações de custo [lista.values]) e y2 (estimações de m2 [lista.values])

```
def elasticidades(modelo, agg, variaveis):
```

Função que calcula a elasticidade das variáveis utilizadas no modelo, sendo *coef beta* * (x/y)

parâmetros:

modelo = recebe o modelo que pode variar de acordo com as variáveis e serviços estimados

agg = tabela agregada

variáveis = variáveis que estarão presentes no modelo

retorna = elast em formato de dicionário com a variável e sua respectiva elasticidade dada pelo modelo, EX: {H_totais: 0,5328}

```
def preco_ideal(modelo, agg, margem_alvo=0.06):
```

Calcula o preço ideal de acordo com a margem de lucro desejada

Parâmetros:

modelo = recebe o modelo que pode variar de acordo com as variáveis e serviços estimados

agg = tabela agregada

margem_alvo = definida como 0.06 mas pode ser alterada na variável Margem no começo do código

```
custo_total_previsto = modelo.fittedvalues.mean()
qtd_media            = agg["Qtd_Contratos"].mean()
custo_por_contrato    = custo_total_previsto / qtd_media
return custo_por_contrato / (1 - margem_alvo), custo_por_contrato
```

```
def imprimir_resultados(servico, m1, m2, y1, y2, agg, variaveis, elast,
cpc, p_ideal):
```

Função que provavelmente o o Well mais possa usar, pois já retorna os valores necessários e seu respectivo enunciado

Chamadas úteis: R2, R2 ajustado, Parâmetros do modelo (coeficientes) e P-valores (significância)

```
print(f" R²           : {m1.rsquared:.4f}")
print(f" R² ajust.   : {m1.rsquared_adj:.4f}")
print(f" F-stat      : {m1.fvalue:.2f} (p={m1.f_pvalue:.5f})")
print(f" MAE         : R$ {mael:,.0f}")
print(f" RMSE        : R$ {rmse1:,.0f}")
print(f"\n Coeficientes:")
for nm, c, p in zip(names, m1.params, m1.pvalues):
    print(f" {nm:24s}: {c:>10,.2f} (p={p:.3f}) {sig(p)}")
```

elasticidades: Ele percorre o dicionário de “elast” item por item, salvando a chave (o nome do componente) na variável nm e o valor (numérico) na variável e

barra = cria uma visualização em barras de acordo com a dimensão de e quanto maior o número, mais barracas terão

senal = identifica se é elasticidade negativa ou positiva

```
for nm, e in elast.items():
    barra = "█" * int(abs(e) * 30)
    sinal = "+" if e >= 0 else "-"
    print(f" ε_{nm:22s}: {e:>+.4f} {barra}")
```

Preficações previstas:

```
print(f"\n — PRECIFICAÇÃO —————")
print(f" Custo médio/contrato : R$ {cpc:,.0f}")
print(f" Preço de breakeven    : R$ {cpc:,.0f} (margem 0%)")
print(f" Preço ideal (6%)      : R$ {p_ideal:,.0f}")
print(f" Preço ideal (10%)     : R$ {cpc/0.90:,.0f}")
```

```
print(f" Preço ideal (15%)      : R$ {cpc/0.85:,.0f}")
```

Por fim, segue um teste com a etapa de validação:

```
agg_val = agregar(df, "Validação")
agg_val["H_Total"] = agg_val["H_Total"] # SDR zerado nessa fase
vars_val = ["Qtd_Contratos", "H_Total", "Num_Mes"]
m1_val, m2_val, y1_val, y2_val = rodar_ols(agg_val, vars_val)
elast_val = elasticidades(m1_val, agg_val, vars_val)
p_ideal_val, cpc_val = preco_ideal(m1_val, agg_val)
imprimir_resultados("Validação", m1_val, m2_val, y1_val, y2_val,
                    agg_val, vars_val, elast_val, cpc_val, p_ideal_val)
```

Resultados atuais:

Expansão

MODELO 1 — Y = Total_Custo _____
R² : 0.0959
R² ajust. : 0.0342
F-stat : 1.56 (p=0.21367)
MAE : R\$ 1,071
RMSE : R\$ 1,270
MODELO 2 — Y = Margem_% _____
R² : 0.9893

Formatação

MODELO 1 — Y = Total_Custo _____
R² : 0.9487
R² ajust. : 0.9452
F-stat : 271.30 (p=0.00000)
MAE : R\$ 913
RMSE : R\$ 1,065
MODELO 2 — Y = Margem_% _____
R² : 0.1114

Validação

MODELO 1 — Y = Total_Custo _____
R² : 0.5915
R² ajust. : 0.5358
F-stat : 10.62 (p=0.00016)
MAE : R\$ 243
RMSE : R\$ 321
MODELO 2 — Y = Margem_% _____
R² : 0.9893

===== Claude generated =====

Documentação técnica — regressao_codigo_final.py

Referência para integração com API Django + SQLite

Constante global

python

ALPHA = 0.10

Nível de significância estatística usado em todos os intervalos de predição. Com `ALPHA=0.10`, o intervalo cobre 90% dos casos esperados (5% em cada cauda). Na API, esse valor seria recebido como parâmetro opcional da requisição — se o cliente não informar, usa o padrão de 10%.

Funções

agregar(df, servico)

O que faz: recebe o DataFrame completo carregado do Excel ou pkl e filtra apenas as linhas do serviço solicitado. Em seguida agrupa por `Mês × Franquia`, somando os valores numéricos e recalculando `Margem_PCT` e `Num_Mes`. Essa agregação é necessária porque a base bruta tem uma linha por contrato — a regressão precisa de uma linha por período.

Parâmetros:

- `df` → `pd.DataFrame` — base completa carregada (Excel ou pkl)
- `servico` → `str` — um dos três valores: "Expansão", "Formatação" ou "Validação"

Retorna: `pd.DataFrame` com as colunas:

Coluna	Tipo	Descrição
<code>Mês</code>	str	Período no formato "2024-01"
<code>Franquia</code>	str	Nome da franquía
<code>Receita_Total</code>	float	Soma da receita do período

<code>Qtd_Contratos</code>	float	Soma de contratos fechados
<code>H_Total</code>	float	Soma de horas (SDR + consultor)
<code>H_Advogado</code>	float	Soma de horas jurídicas
<code>Total_Custos</code>	float	Soma dos custos alocados
<code>Margem_RS</code>	float	Receita – Custos
<code>Margem_PCT</code>	float	$Margem_RS / Receita_Total$
<code>Num_Mes</code>	int	Número do mês (1 a 12)

Na API Django: chamada uma única vez no `AppConfig.ready()` para cada serviço, gerando `agg_exp`, `agg_fmt` e `agg_val` em memória. Quando novos dados chegarem via `POST /dados`, o sinal `post_save` do Django rechamaria `agregar` para atualizar o DataFrame antes de retreinar o modelo.

`rodar_ols(agg, variaveis)`

O que faz: recebe o DataFrame agregado de um serviço e a lista de variáveis independentes, monta a matriz X adicionando o intercepto manualmente via `np.ones`, e ajusta dois modelos OLS — M1 com `Total_Custos` como alvo e M2 com `Margem_PCT`. Retorna ambos prontos para uso.

Parâmetros:

- `agg` → `pd.DataFrame` — saída de `agregar()`
- `variaveis` → `list[str]` — nomes das colunas X, por exemplo `["Qtd_Contratos", "H_Total", "Num_Mes"]`

Retorna: tupla (`m1`, `m2`, `y1`, `y2`):

Elemento	Tipo	Descrição
<code>m1</code>	<code>RegressionResultsWrapper</code>	Modelo de custo — $Y = Total_Custos$
<code>m2</code>	<code>RegressionResultsWrapper</code>	Modelo de margem — $Y = Margem_PCT$

y1	<code>np.ndarray</code> (float)	Vetor observado de Total_Custos
y2	<code>np.ndarray</code> (float)	Vetor observado de Margem_PCT

Os objetos `m1` e `m2` expõem os atributos mais relevantes para a API:

Atributo	Tipo	Uso na API
<code>.params</code>	<code>np.ndarray</code>	Coeficientes $\beta_0, \beta_1, \dots, \beta_k$
<code>.pvalues</code>	<code>np.ndarray</code>	Significância de cada coeficiente
<code>.rsquared</code>	<code>float</code>	R^2 do modelo
<code>.rsquared_adj</code>	<code>float</code>	R^2 ajustado
<code>.fvalue</code>	<code>float</code>	F-statistic
<code>.f_pvalue</code>	<code>float</code>	p-valor do F
<code>.fittedvalues</code>	<code>np.ndarray</code>	Valores ajustados ($\hat{y}$)
<code>.get_prediction(x)</code>	método	Gera intervalos de predição

Na API Django: chamada no startup junto com `agregar`. Os objetos `m1` e `m2` de cada serviço ficam armazenados em memória (variáveis de módulo ou em `AppConfig`) e são reutilizados em cada requisição sem retreinar.

`elasticidades(modelo, agg, variaveis)`

O que faz: calcula a elasticidade pontual de cada variável X avaliada na média dos dados, usando a fórmula $\epsilon_{xi} = \beta_i \times (X_i / \bar{Y})$. O índice i percorre a lista `["const"] + variaveis` para alinhar corretamente com `modelo.params`.

Parâmetros:

- `modelo` → `RegressionResultsWrapper` — saída de `rodar_ols` (use sempre `M1`)
- `agg` → `pd.DataFrame` — DataFrame agregado do serviço
- `variaveis` → `list[str]` — mesma lista usada no ajuste

Retorna: `dict[str, float]` — chaves são os nomes das variáveis, valores são as elasticidades:

```
python
# Exemplo para Formatação
{
    "Qtd_Contratos": 0.2417,
    "H_Total":      0.1350,
    "H_Advogado":   0.3703,
    "Num_Mes":      0.0113
}
```

Interpretação: $\epsilon_{H_Advogado} = 0.37$ significa que um aumento de 1% nas horas de advogado eleva o custo total em 0.37%, mantendo as demais variáveis fixas.

Na API Django: endpoint `GET /api/elasticidades/<servico>/` retorna esse dicionário diretamente serializado como JSON. Alimenta os gráficos de curva de elasticidade do dashboard.

`preco_ideal(modelo, agg, margem_alvo)`

O que faz: calcula o preço por contrato necessário para atingir a margem alvo, usando o custo médio histórico previsto pelo modelo. Divide `fittedvalues.mean()` pela quantidade média de contratos para obter o custo por contrato, depois aplica a fórmula `Preço = CPC / (1 - margem_alvo)`.

Parâmetros:

- `modelo` → `RegressionResultsWrapper` — M1 ajustado
- `agg` → `pd.DataFrame` — DataFrame agregado do serviço
- `margem_alvo` → `float` — padrão `0.06` no seu código (6%)

Retorna: tupla (`preco_ideal`, `custo_por_contrato`), ambos `float` em reais.

Observação importante: esse retorno representa o custo médio histórico, não uma projeção para um cenário futuro. Para precificação prospectiva, use `simular()` com os inputs reais do próximo contrato.

Na API Django: alimenta o endpoint `GET /api/diagnostico/<servico>/` como parte do resumo do modelo, junto com R^2 e coeficientes.

`imprimir_resultados(servico, m1, m2, y1, y2, agg, variaveis, elast, cpc, p_ideal)`

O que faz: função de apresentação — imprime no terminal o diagnóstico completo de M1 e M2, incluindo R^2 , coeficientes com p-valores e símbolo de significância, elasticidades com barra visual, equação estimada formatada e tabela de precificação.

Parâmetros: todos os resultados das funções anteriores reunidos.

Retorna: `None` — apenas imprime.

Na API Django: não vai para a API diretamente. Toda a lógica interna dela — cálculo de `mae`, `rmse`, formatação dos coeficientes — seria extraída e transformada em um dicionário de retorno JSON. A estrutura do endpoint `GET /api/diagnostico/<servico>/` seria:

json

```
{
  "servico": "Formatação",
  "n_observacoes": 48,
  "m1": {
    "r2": 0.9490,
    "r2_ajustado": 0.9440,
    "f_stat": 176.3,
    "f_pvalue": 0.00001,
    "mae": 2174,
    "rmse": 2631,
    "coeficientes": {
      "const": {"beta": 9307.0, "pvalue": 0.005, "sig": "***"},
      "Qtd_Contratos": {"beta": 7400.0, "pvalue": 0.001, "sig": "***"},
      "H_Total": {"beta": -22.45, "pvalue": 0.760, "sig": "ns"},
      "H_Advogado": {"beta": 686.0, "pvalue": 0.000, "sig": "***"},
      "Num_Mes": {"beta": 274.62, "pvalue": 0.021, "sig": "**"}
    }
  },
  "m2": {
    "r2": 0.680,
    "margem_media_obs": 0.445
  },
  "precificacao": {
    "custo_por_contrato": 30617,
    "preco_breakeven": 30617,
    "preco_ideal_6pct": 32571,
    "preco_ideal_10pct": 34019,
    "preco_ideal_15pct": 36020
  },
  "elasticidades": {
    "Qtd_Contratos": 0.2417,
    "H_Total": 0.1350,
    "H_Advogado": 0.3703,
    "Num_Mes": 0.0113
  }
}
```



```
}
```

`simular(modelo, variaveis, inputs, margem_alvo, alpha)`

O que faz: é o coração da ferramenta de precificação. Monta o vetor de inputs com intercepto, chama `modelo.get_prediction(x_vec).summary_frame(alpha)` e extrai os intervalos de predição. Divide todos os valores pela quantidade de contratos e calcula o preço ideal para cada ponto do intervalo.

Parâmetros:

- `modelo` → `RegressionResultsWrapper` — M1 ajustado do serviço
- `variaveis` → `list[str]` — mesma lista usada no ajuste
- `inputs` → `dict[str, float]` — valores do cenário, **sem o intercepto**:

python

```
{"Qtd_Contratos": 1, "H_Total": 150, "H_Advogado": 12, "Num_Mes": 13}
```

- `margem_alvo` → `float` — padrão `0.06`
- `alpha` → `float` — padrão `ALPHA = 0.10`

Retorna: `dict` com 9 chaves, todas `float`:

Chave	Descrição
<code>custo_central</code>	Custo total — estimativa pontual
<code>custo_lower</code>	Custo total — limite inferior (intervalo de predição)
<code>custo_upper</code>	Custo total — limite superior (intervalo de predição)
<code>cpc_central</code>	Custo por contrato — central
<code>cpc_lower</code>	Custo por contrato — inferior
<code>cpc_upper</code>	Custo por contrato — superior
<code>preco_central</code>	Preço ideal — central
<code>preco_lower</code>	Preço ideal — inferior

`preco_upper` Preço ideal — superior

Na API Django: é o endpoint principal — `POST /api/preco/`. O body da requisição mapeia diretamente para os parâmetros da função, e o dicionário retornado vai direto para o `JsonResponse`:

```
python
# views.py
from django.http import JsonResponse
from django.views import View
import json

class PrecoView(View):
    def post(self, request):
        body = json.loads(request.body)
        servico = body.get("servico") # "Expansão" | "Formatação" | "Validação"
        inputs = body.get("inputs") # dict com as variáveis
        margem = body.get("margem_alvo", 0.06)
        alpha = body.get("alpha", 0.10)

        # Seleciona o modelo e variáveis do serviço solicitado
        modelo, variaveis = MODELOS[servico] # dict carregado no startup

        resultado = simular(modelo, variaveis, inputs, margem, alpha)
        return JsonResponse(resultado)
```

Mapa de endpoints → funções

Endpoint	Método	Função(ões)
<code>GET /api/diagnostico/<servico>/</code>	GET	<code>rodar_ols + elasticidades + preco_ideal</code>
<code>POST /api/preco/</code>	POST	<code>simular</code>
<code>GET /api/elasticidades/<servico>/</code>	GET	<code>elasticidades</code>
<code>POST /api/dados/</code>	POST	salva no SQLite → sinal <code>post_save</code> → <code>agregar</code> → <code>rodar_ols</code>

Ciclo de vida na API Django

startup (AppConfig.ready())

- └─ lê Excel ou pkl
- └─ agregar() × 3 serviços → agg_exp, agg_fmt, agg_val
- └─ rodar_ols() × 3 serviços → modelos em memória

requisição POST /api/preco/

- └─ simular(modelo_em_memoria, inputs_do_body)
- └─ retorna JSON com central + lower + upper

requisição POST /api/dados/ (novos dados chegam)

- └─ salva no SQLite
- └─ post_save signal dispara
- └─ agregar() + rodar_ols() recalibram o modelo em memória
- └─ próxima requisição já usa os coeficientes atualizados

○